ChatGPT

Top ML Models for Short-Term Price Prediction from Liquidation Data

Predicting near-term price direction from high-frequency **liquidation data** requires models that can capture rapid order-flow dynamics. Below are three top machine learning models known to perform well in high-frequency or microstructure contexts, followed by techniques to make backtests more robust and realistic.

1. Recurrent Neural Networks (LSTM/GRU Variants)

Why it's suitable: Recurrent neural networks, especially Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), excel at sequence modeling. They can learn patterns from the **temporal order flow** – for example, sequences of liquidations and price responses – to predict short-term price moves. LSTMs have shown strong performance on financial time series, consistently outperforming many alternatives in high-frequency price prediction tasks [1]. In crypto and stock experiments, LSTM-based models demonstrated robust accuracy for intraday price direction forecasting, often beating Transformer-based models on short-horizon predictions and proving more **practical for trading** (e.g. handling transaction costs and noise) [1].

Microstructure performance: Research indicates that LSTMs (and BiLSTMs) can capture complex patterns in **market microstructure** data. For instance, an LSTM combined with CNN layers (the DeepLOB architecture) achieved state-of-the-art results on high-frequency limit order book data [2]. This hybrid model could identify subtle order flow imbalances and short-term trends, validating that recurrent networks can extract predictive signals from rapid trade/liquidation sequences. Overall, RNNs are adept at modeling **event timing and autocorrelation** in liquidation events (e.g. bursts of stop-loss liquidations), which often precede price reversals or momentum.

Implementation notes: - *Data formatting:* Feed the model a sequence of features for each short time window. For example, create time-series features such as the count and volume of buy vs. sell liquidations per second, price changes, etc., over the last N minutes. An LSTM can ingest a rolling window of these high-frequency observations and output a prediction for the next interval's price direction (up/down). - *Architecture:* Use a small stack of LSTM or GRU layers (with possible dropout regularization) to capture temporal dependencies. For instance, a 2-layer LSTM with hundred of units can learn short-term patterns. You may also experiment with **bidirectional LSTMs** (to capture patterns in both directions) or an attention mechanism if looking to emphasize certain event sequences. - *Training:* Train the network to classify the next price movement (e.g., up vs. down in the next 5-minute window) using a labeled dataset of historical windows. Include early stopping and dropout to prevent overfitting. Ensure to do time-order validation (no shuffling) so the model generalizes to truly future data.

2. Deep Convolutional Networks (DeepLOB-Style CNN)

Why it's suitable: Convolutional neural networks can effectively detect local **spatiotemporal patterns** in order flow data. In the context of liquidations, a CNN can identify patterns like sudden spikes in liquidations or clustering of buy/sell liquidations that often precede price jumps. CNN-based models have been very successful in **high-frequency price prediction**. A prime example is the **DeepLOB** model, which uses convolutional blocks on limit order book data and *outperformed all existing state-of-the-art algorithms* on a standard LOB benchmark [3]. This CNN+LSTM architecture also delivered stable out-of-sample accuracy on real-world data, showing it could extract robust features from microstructure inputs [3]. The convolution layers act as feature extractors, filtering the noisy high-frequency data to pick out predictive signals (e.g. a flurry of long-liquidations indicating potential price rebound).

Microstructure performance: CNNs are known to excel at recognizing shapes or bursts in data. In trading, researchers have even **converted order flow data into images** so that CNNs can detect patterns of activity [3]. The DeepLOB approach treats a snapshot of the order book (or recent order flow) as a matrix and uses 1D/2D convolutions to capture the "shape" of supply-demand imbalance preceding a move. For liquidation data, a CNN could catch a signature like *"rapid succession of sell-liquidations of increasing size"* which might signal a final capitulation before price upticks. Such models have proven their worth in *HFT contexts* by quickly responding to microstructure signals. Zhang et al. (2019) showed that adding convolutional filters on top of LSTM (i.e. DeepLOB) significantly improved short-term prediction accuracy in volatile, high-frequency environments [2].

Implementation notes: - *Input representation:* Construct a time-window matrix of recent liquidation activity. For example, use the past few minutes divided into seconds (rows) and features (columns like buy-liquidation volume, sell-liquidation volume, etc.). This forms an "image" of liquidation intensity over time. A 1-D CNN can slide over the time axis, or a 2-D CNN can be used if you stack multiple features as channels.
- *Architecture:* Use multiple convolutional layers to capture patterns at different scales. For instance, **Temporal Convolutional Networks (TCN)** or standard 1D CNN layers with various filter lengths can detect short bursts vs. longer waves of liquidations. In DeepLOB, several convolutional blocks (including Inception modules) were used before handing off to an LSTM [2] – you can simplify this by using a few Conv layers followed by either an LSTM or a dense layer for classification. The convolution filters will act like detectors for events such as a cluster of large liquidations. - *Training:* Similar to the LSTM, train the CNN to classify the next price move using labeled data. Use techniques like batch normalization and dropout in the conv layers to generalize better. CNNs are less likely to overfit small temporal shifts, which is useful – e.g., a pattern of 10 large liquidations is predictive regardless of slight timing, and convolution handles that naturally via sliding filters.

3. Gradient Boosted Decision Trees (XGBoost/LightGBM)

Why it's suitable: Gradient boosted tree ensembles (like **XGBoost** or LightGBM) are powerful for structured **tabular data**, and they have been successfully applied to short-term market prediction. These models handle non-linear interactions of features and are relatively robust against overfitting due to built-in regularization (shrinkage, tree depth limits). In intraday trading research, XGBoost-based models have achieved strong accuracy in predicting price direction [4]. For example, Huang et al. used an XGBoost classifier to predict intraday trend changes and reported **satisfactory forecasting performance** [4]. XGBoost's advantage is that it can ingest a rich set of engineered features from the liquidation data (counts, volumes, ratios, recent returns) and find complex patterns (decision rules) that correlate with short-horizon

price moves [5]. It's also fast to train and evaluate – a big plus in high-frequency contexts where retraining or rapid inference may be needed.

Microstructure performance: While tree models don't model sequences natively, they excel when you summarize the sequence into informative features. Order flow studies have used features like order imbalance, volatility, and event frequencies, and tree ensembles often outperform simpler models on these. XGBoost in particular has been noted to **improve prediction accuracy** and handle large datasets efficiently [5]. Researchers have found it superior to basic machine learning baselines in many financial classification tasks [4]. In practice, you might compute features per 5-minute bin (e.g., number of long liquidations, total liquidated volume, % of volume that was liquidations, price change during the window, etc.) and let the boosted trees figure out threshold combinations (like "if many long liquidations and price didn't fall much, expect a bounce"). Tree models are also relatively interpretable; feature importance can highlight which aspects of liquidation activity are most predictive of the price direction.

Implementation notes: - *Feature engineering:* Aggregate the raw tick-level data into features for each time window. Possible features include: total count of liquidations (and split by buy vs sell), sum of notional liquidated, largest single liquidation event, recent price momentum, order book imbalance (if available), time since last large liquidation, etc. Incorporate a few lagged features (from previous intervals) to give the model context of recent trends. - *Model training:* Use XGBoost or LightGBM's classifier on these features to predict the next period's price movement (up/down/stationary). Keep the tree depth moderate (e.g. 4–8) to avoid overfitting, and use cross-validation to tune hyperparameters like learning rate and number of trees. These ensembles have parameters like `gamma` (min gain) or `lambda` (L2 regularization) which can further regularize the model. - *Performance and update:* Thanks to their speed, boosted trees can be retrained frequently as new data comes in (even online). They also handle **class imbalance** well by boosting residual errors – if needed, you can adjust the classification threshold or apply techniques like SMOTE as in some studies, but often ensuring enough data for both up/down moves and using proper eval metrics (AUC, etc.) suffices.

Techniques for Robust Backtesting and Overfitting Reduction

Even a great model can lead to misleading backtest results if we assume perfect trade execution or fit too closely to noise. Here are several techniques to **regularize the model and make backtests more realistic**:

- **Slippage & Transaction Cost Modeling:** Always account for execution slippage and fees in your strategy. For example, assume you **enter at a worse price** than the theoretical signal. A common practice is adding a tick or two of slippage on each trade and using the average spread plus a buffer in backtests [6]. This means if a liquidation spike suggests a long entry at \$1000, you might model your fill at \$1000 + 0.1% (or a couple of ticks higher) to reflect real market impact. By incorporating even a minimal cost buffer, you avoid strategies that only work on **idealized prices**. (Failing to do this often yields overly optimistic results [6].) In short, bake realistic frictions into the backtest – many ostensibly profitable high-frequency signals vanish once you subtract the bid-ask spread or slippage.

- **Signal Buffering and Confirmation:** Similar to slippage modeling, introduce a **buffer or delay for trade signals** derived from liquidations. Liquidation events cause sharp, fast moves that may revert quickly; it's often unrealistic to catch the exact bottom or top tick. To make your strategy more robust, require a confirming price movement or a time delay before acting on a signal. For instance,

if your model predicts a price rise after a flush of sell-liquidations, you could wait a few seconds or until price ticks up a small amount before buying. This ensures the move wasn't just a blip and reduces sensitivity to untradeable sudden spikes. Essentially, don't assume you can buy at the liquidation's exact trigger price – **wait for stabilization** or trade at a slightly less favorable price. This buffered approach yields more conservative (but realistic) entries and will show if the alpha survives minor execution lags.

- **Regularization & Simpler Models:** To prevent overfitting, apply classic regularization both in modeling and backtest design. In modeling, prefer simpler architectures or add penalties: e.g. use dropout layers in neural nets, limit tree depths in boosting, or include an L_1/L_2 penalty to discourage overly complex patterns. Simpler models tend to be more robust on new data, which is crucial in non-stationary markets. Also, ensure your **training regime prevents lookahead bias** – use walk-forward or purged cross-validation (leaving a gap between train and test periods) so the model doesn't subtly learn future info. Lopez de Prado's purging/embargo techniques are good practice to get an honest measure of performance. If the model is truly finding structural patterns (and not just noise), it should perform well on a forward test with no overlap.

- **Backtest Validation & Cost-aware Targets:** Incorporate realism into what the model is predicting. One approach is to **adjust your target or entry rules to include costs**. For example, label a trade as "win" only if the price moves more than a certain threshold (covering fees/spread) in your favor; small moves that wouldn't overcome transactions costs should be treated as no signal. This trains the model to focus on **meaningful moves** and avoids overfitting to tiny fluctuations. Moreover, evaluate your strategy on metrics that include cost (net P&L after fees) rather than just direction accuracy. A cautionary example from literature: a model might predict the mid-price direction correctly, but if it doesn't cover the spread, the strategy isn't profitable [7]. As Kearns & Nevmyvaka note, the mid-price is a "fictitious, idealized price" – once you account for real trading costs, a lot of apparent edge can disappear [7]. So, regularize your backtest by **demanding a margin above noise/cost** for a signal to count and by stress-testing the strategy with different slippage assumptions.

By using the appropriate high-frequency models (like RNNs, CNNs, and boosted trees) and enforcing these buffering and regularization techniques, you increase the likelihood that your discovered "alpha" is real and achievable. The combination of a powerful model and a **rigorously realistic backtest** will yield short-term trading signals that are both statistically significant and practically tradeable.

Sources: Machine learning for microstructure data [2] [3]; LSTM vs. Transformer performance in HFT [1]; XGBoost in intraday forecasting [4]; backtest cost buffers [6]; importance of accounting for trading costs [7].

---

[1] [2] openreview.net
https://openreview.net/pdf/06f4232517e3c80aef7d6c683719114e1f037413.pdf

[3] [1808.03668] DeepLOB: Deep Convolutional Neural Networks for Limit Order Books
https://arxiv.org/abs/1808.03668

4 5 High-Frequency Direction Forecasting of the Futures Market Using a Machine-Learning-Based Method
https://www.mdpi.com/1999-5903/14/6/180

6 How to Backtest Trading Strategies on TradeStation (Step-by-Step) | QuantVPS
https://www.quantvps.com/blog/how-to-backtest-on-tradestation?srsltid=AfmBOor2aqQDLpEGsRrk92eP3wRzrK4i3jPgNV6EeJrDGUvV-F_KTfRm

7 cis.upenn.edu
https://www.cis.upenn.edu/~mkearns/papers/KearnsNevmyvakaHFTRiskBooks.pdf